

## Experiment Number:4

**Aim:**Develop a program to demonstrate parameter passing techniques

- 1.Call by Value
- 2.Call by Reference
- 3.Call by Address)

### Code:

```
#include <bits/stdc++.h>
using namespace std;
```

```
//1 Call by value , changes here do not get reflected in the original variables
```

```
// A copy of the original variable is passed
```

```
void val1(int a){
    a=20;
    cout<<"Variable inside callbyvalue fuction:"<<a<<endl;
}
```

```
//2 Call by reference , changes here do get reflected
```

```
// A reference of the original variable is passed
```

```
void val2(int &a){
    a=30;
    cout<<"Variable inside callbyreference fuction:"<<a<<endl;
}
```

```
//3 Call by address , changes here do get reflected
```

```
// Memory pointer of the original variable is passed
```

```
void val3(int *a){
    *a=40;
    cout << "Variable inside callbyaddress fuction: " << *a<<endl;
}
```

```
int main() {
    // Develop a program to demonstrate parameter passing techniques
    //1 call by value
    //2 call by reference
    //3 call by address
    //and analyze their effect on variable modification

    // 1. Call by Value
    int x = 10;
    cout << "Before callbyvalue: " << x << endl;
    val1(x);
    cout << "After callbyvalue: " << x << endl;

    int y = 10;
    cout << "Before callbyreference: " << y << endl;
    val2(y);
    cout << "After callbyreference: " << y << endl;

    // 3. Call by Address
    int z = 10;
    cout << "Before callbyaddress: " << z << endl;
    val3(&z);
    cout << "After callbyaddress: " << z << endl;
    return 0;
}
```

## Output:

### Your Output

```
Before callbyvalue: 10
Variable inside callbyvalue  fuction:20
After callbyvalue: 10
Before callbyreference: 10
Variable inside callbyreference fuction:30
After callbyreference: 30
Before callbyaddress: 10
Variable inside callbyaddress fuction: 40
After callbyaddress: 40
```